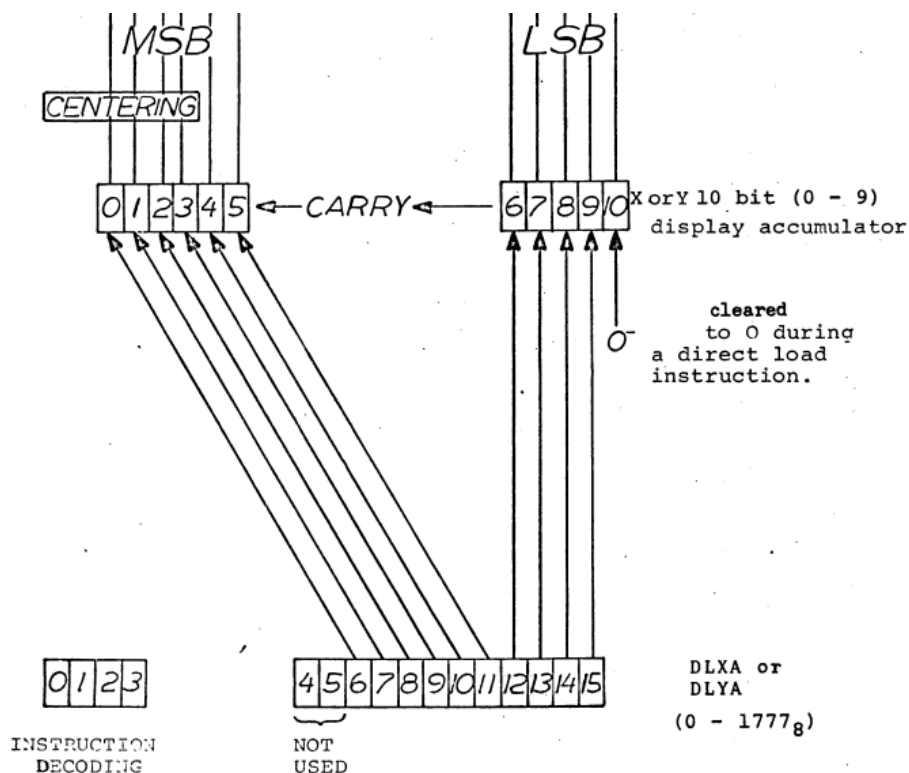


RECAP: How the mature PDS-1 and PDS-1D work

The standard way that the screen positions are addressed and tracked in most surviving IMLAC software (and in the slmlac emulator) is described in the [Programming Guide](#) for the PDS-1D. There are two registers, the X and Y accumulators, that track the 2D position of the beam on the screen. The beam position can be updated by using a D LXA or D LXY instruction to load a new 12-bit screen address into one of the accumulators. However, the two most significant bits are ignored, so in practice it is really a 10-bit address. Therefore, valid addresses range from 0 to 1777_8 , with 1000_8 representing the center of the screen.

The upper 6 bits of the input are sent to the Most Significant Bit (MSB) deflection amplifier that controls the coarse positioning of the beam. The lower 4 bits are sent to the Least Significant Bit (LSB) amplifier that controls the fine positioning. The MSBs essentially divide the screen into a 64×64 grid, and within each square, the LSBs form a 16×16 subgrid. Together they form a 1024×1024 grid of addressable positions.

The lower 4 bits may be shifted left or right, based on a scale factor that is set using the D STS instruction. This allows drawing operations to occur on half-grid points, so you can also think of the screen resolution as being 2048×2048 . When scale is set to 1, the MSBs and LSBs act like a continuous 10-bit register. (Freeway stays in scale=1 throughout the entire program, so I won't discuss that aspect of the display routines again until the end of this document.)



The Problem

It's clear that Freeway uses a very different screen addressing method than the one described above. Most of D LXA and D LYA instructions in the code have input values well outside the valid range. Explicit literal values written in the assembly code range from 1400₈ to 6000₈, and some self-modifying code routines can produce D LXA instructions with inputs as high as 6400₈. This explains why the graphics of Freeway look so chaotic when played on the slmlac emulator. The first two bits of each of these position inputs are being silently thrown away, and the remaining bits are being interpreted as if they were using the standard form. None of the vectors are being drawn in the right place.

The Reason

Why does Freeway use a unique method of addressing the screen? I think the reason is described on [page 57 of the Programming Guide](#): the distinction between “Graphics machines” and “Alpha machines”. The PDS-1D and “later model” PDS-1 machines are Graphics machines, which use the standard documented approach. But “early model” PDS-1s are Alpha machines, where screen addressing worked somewhat differently.

The exact operational details of Alpha machines are not fully described, but the main difference given is that there was no arithmetic carry between the Most and Least Significant Bits (MSBs and LSBs) of the deflection amplifiers. So even under scale=1, an Alpha machine could not treat the screen positions as a single 10-bit accumulator. Increment mode vector drawing could only occur within a bounding box set by the MSBs, and carry between the LSBs and MSBs had to be handled manually in software.

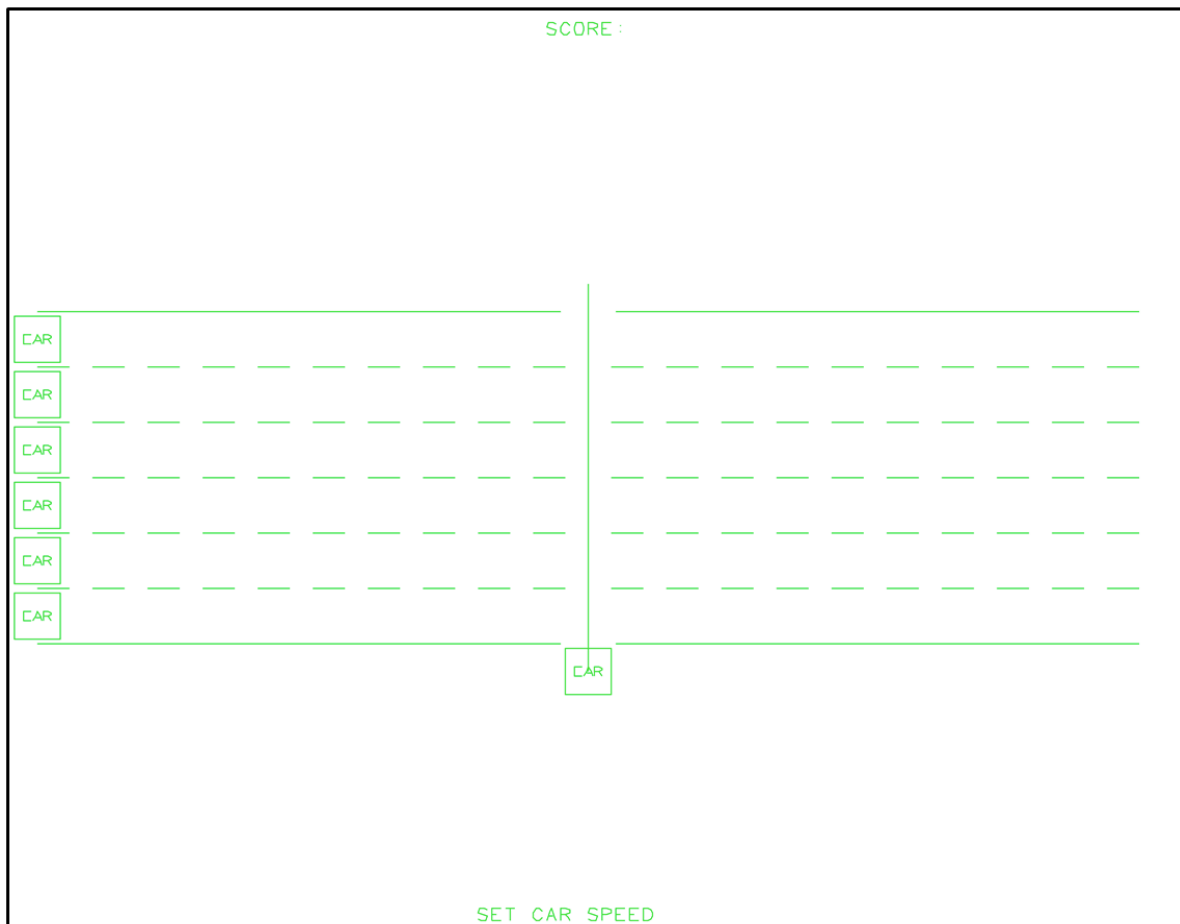
It seems clear based on code analysis that Freeway was written for one of these Alpha machines. It contains explicit code in many places to update the MSBs separately from the vector increments. The timeline also makes sense. The author of Freeway says that it was [written in 1971](#), and a description of the University of Washington computer lab written by his supervisor, Dr. Earl Hunt, and [published in Jan 1970](#) says “we are now attaching an IMLAC Corp. PDS-1 Display Unit”. Since the PDS-1 wasn't released until late 1969, this must have been from a fairly early batch of units.

However, the lack of carry between the LSBs and MSBs is not the only difference. The screen addresses in Freeway are being interpreted in a fundamentally different way, which is why the input values are usually outside the “standard” range of 0 to 1777₈. Unfortunately, no detailed description of how the Alpha machines work has survived to be put online, as far as I can find. But there are enough contextual clues within the code of Freeway that I think it is possible to reconstruct how an Alpha machine (or at least, how that *particular* Alpha machine) addresses the screen.

How Freeway Is Supposed to Look

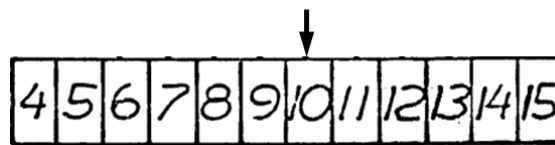
The goal of Freeway is to drive a car through an intersection while avoiding six lanes of cross-traffic. Your car cannot steer and must travel at a constant speed without stopping. Before each round, you can choose your car's speed (from 1 to 9) using the up or down keys. You then press the XMIT button to begin the round. The 6 other cars (one in each lane) begin traveling across the screen at different speeds. You wait for the right moment, then press XMIT again to "FIRE" your car through the intersection at your chosen speed. You either make it through all six lanes or there's a crash. It's essentially "ballistic Frogger".

The drawing routine on each frame is pretty straightforward. First it snakes back and forth down the screen drawing 7 long horizontal lines that represent the highway. There are solid lines top and bottom representing the edges of the road, and 5 dashed lines in the middle acting as lane separators. A gap in the center of the screen provides a path for the player car. A single solid vertical line is then drawn upward through the gap, showing the player's trajectory. A scoreboard is printed at the top of the playfield and a status message is printed at the bottom. Finally, the player car and the six other cars are drawn in their respective positions. (Each car looks like a square with the word "CAR" written inside it.) Overall, the final screen layout at startup should look like this:



How Screen Addressing Works in FREEWAY

So, how do you determine the actual screen addressing method that was used, especially without already knowing in advance how the screen is supposed to look? The first clue came from the spacing of the computer cars. I knew there were 6 cars and that the game was supposed to bear some similarity to the game Frogger, so I figured that the 7 long horizontal lines probably drew a roadway with 6 lanes. The Y-positions of the 6 cars are hardcoded from 3300_8 to 4500_8 in intervals of 200_8 . Meanwhile, the top line of the freeway has a Y-position of 4600_8 and subsequent lines are positioned by calling D LYA to decrement the Ymsb 4 times. If we assume that the cars are centered in their lanes, then the spacing of the lines must also be 200_8 , so a single increment of the MSBs must change the screen position by 40_8 . In other words, they increment bit 10 instead of bit 11.



From this, it seems most likely that the LSBs are stored somewhere within bits 11-15 and the MSBs are stored within bits 4-10. Not all of the bits are necessarily used (just as bits 4 and 5 are unused in the final standard). My first thought was that no underlying behavior had changed, but that unused bits had simply been redistributed so that it was basically just a change of formatting. Maybe the extra bits were being used as link bits to help the programmer manually keep track of carry operations between the LSBs and MSBs, and they became vestigial once that process was made automatic? This seemed promising, but none of the combinations I tried produced any coherent drawings. There had to be something more going on.

The solution came from the CONVERT subroutine:

```
222                                88 CONVERT BSS 1 /CONVERT TO D LXA
223      003061                    89 SAR 1
224      022722                    90 DAC #TMP
225      004011                    91 LAW 11
226      020231                    92 DAC .+3
227      062722                    93 LAC #TMP
230      035214                    94 JMS D
231                                95 BSS 1
232      003043                    96 SAL 3
233      003042                    97 SAL 2      /GET MSB PORTION
234      064231                    98 ADD .-3      /ADD LSB PORTION
235      066660                    99 ADD =11400 /MAKE D LXA
236      110222                   100 I JMP CONVERT /RETURN
```

Each car's X-position is stored as an internal value that can range from 0 to 2640₈. The job of CONVERT is to take this value and convert it into a complete 16-bit D LXA instruction that is then written into the display routine (directly overwriting the machine code from the previous frame) to change the starting position where a car is drawn. The steps for this are:

- Shift right by 1 (effectively dividing the position value by 2)
- Integer divide the result by 9 (using the "D" subroutine), getting both a quotient and a dividend.
- Shift the quotient left by 5 (moving the MSBs into the correct place)
- Add the dividend (Putting the LSBs in the correct place)
- Add 11400 (a "magic" number that creates the D LXA instruction and also increases the MSBs so that an internal position value of "0" corresponds to the left edge of the playfield).

Due to the initial right shift, the 2640₈ possible internal position values map to a range of 1320₈ (i.e. 720) possible screen positions that should ideally all move smoothly from one end of the playfield to the other.

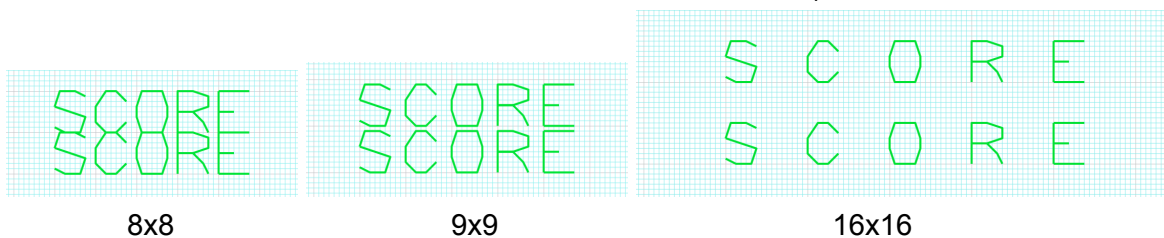
The really surprising part of this is the divide by 9 to split the value into MSB and LSB portions. In other words, it's using a 9x9 subgrid for the LSBs instead of 16x16. I thought I was misinterpreting this for quite a while, assuming that it had to actually be 8 or 16 subunits (using 3 or 4 bits for the LSBs). But when you [work out the steps explicitly](#), it's clear. The first 10 distinct screen positions count up like this:

- 0001 0011 0000 0000
- 0001 0011 0000 0001
- 0001 0011 0000 0010
- 0001 0011 0000 0011
- 0001 0011 0000 0100
- 0001 0011 0000 0101
- 0001 0011 0000 0110
- 0001 0011 0000 0111
- 0001 0011 0000 1000
- 0001 0011 0010 0000 /HUH?

This is surprising in the way that it uses but mostly "wastes" bit 12. IMLAC could have used the 4 bits to achieve a sub-resolution of 16x16, so why would they choose to cap it at 9x9? It also looks tempting to think that bit 12 could be intended as a link bit to indicate when the first 3 bits overflow. I stayed fixated on that interpretation for quite a while. I thought I was misinterpreting how the bit math in the CONVERT subroutine really works. Or maybe the programmer had been a little confused but had still managed to make something a little janky but workable. But no, that's really how it works!

Once you embrace the combination of *BOTH* a 9x9 LSB subgrid and no automatic carry between the LSBs and MSBs, the graphics suddenly start making coherent sense in so many different ways:

- The solid-line drawing subroutines that draw the road edges (“LR5”) and the vertical trajectory line (“UP”) work by entering increment mode, drawing six 3-unit-long vector increments for a total distance of 18, then clearing the LSB and incrementing the MSB by 2. With a 9x9 subgrid, the two MSB increments also equal 18 units, so the result when calling LR5 or UP multiple times in a row is to draw a solid, continuous line across the screen.
- The dashed lines (lane separators) work by drawing seven 3-unit vectors, then incrementing the MSBs 4 times (shifting 36 units) resulting in a dashed line of “21 on, 15 off”. The dashed lines are either drawn to the left (subroutines “LLD2” called x10, then “LLD4” x10) or to the right (“LRD1” x10 then “LRD3” x10). The two different forms of each subroutine offset the dashes differently within the 36-unit interval, which creates the gap in the middle. While this would always produce some sort of dashed line, it’s only when you embrace both the 9x9 subgrid *and* no automatic MSB increments that these dashed lines are
 - identical when drawn right or left,
 - are the exact same length as the solid lines, and
 - have the middle gap in the right place.
- All of the car positions increment smoothly across a range that also perfectly matches the length of the highway lines.
- The text on the screen was clearly written in a vector “font” that expects a 9x9 subgrid. Each letter fits into a box 6 units wide and 8 units tall resulting in reasonable looking kerning and tight-but-readable line spacing if you were to write multiline text. If you use these same letter forms on later Imlac hardware with a 16x16 grid, the letters would end up too widely spaced. Admittedly, if you assume an 8x8 subgrid then the kerning still looks okay, but then multiline text would bleed together vertically. (For example, you wouldn’t be able to tell a F over an E from an E over an E.)



Based on all this evidence, it’s clear that 9-units of LSB, stored in bits 12-16 are the way to go. But in order to implement this in an emulator in a complete and coherent way, there are still some lingering questions...

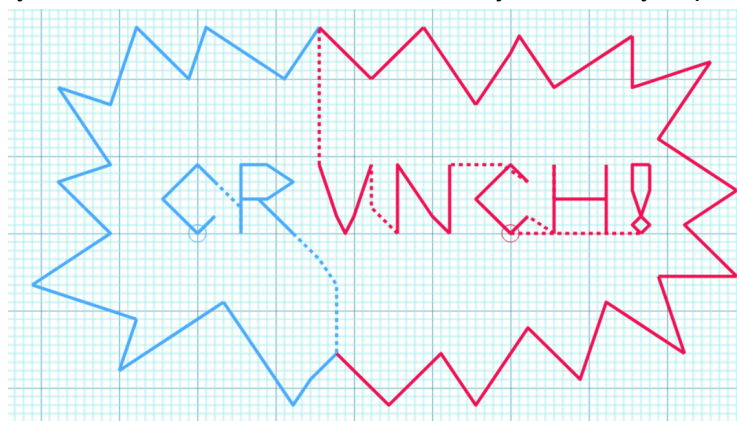
Question: How big is the “bounding box” in Increment Mode?

This is a tricky one. One might naively think that increment mode would only work within a single 9x9 subgrid before the MSB had to be incremented manually, but that’s clearly not the case in any of the drawing routines. For example, the standard line drawing routines draw up to 21 units from the starting point in increment mode before resetting and incrementing the MSBs to the next position. And, as a design decision, it would be pretty constraining for the programmer. You almost might as well not even *have* an increment mode if you have to stop and reset every 9 units.

So perhaps that’s what the extra bits in the address are for. Maybe you can use bits 11-15 to keep track of your LSB position, meaning you could move 32 units from your starting point (a little more than 3 major grid points) before you have to reset and increment the MSBs. In other words, there could be multiple valid addresses for each screen position where the bounding boxes from each MSB grid point overlap. That makes some sense as a “natural” design constraint of the hardware, and it is true that in FREEWAY the beam never travels more than 26 units away from its starting point. But that still doesn’t quite work because the beam can move in either a positive or negative direction from the starting point. That means the LSB quantity would have to be signed, and the beam frequently moves beyond a limit of ± 16 implied by only having 5 bits to work with.

So maybe there is no limit on how far you can go in increment mode, and the screen addresses are just used to set the initial position of the beam. Maybe the original hardware didn’t even have dedicated registers tracking the screen position, and these addresses are just static inputs for the D L_A instructions. It’s possible this is true, but it seems pretty likely there was still some limit to how far increment mode could travel from the starting position. For one thing, the [Programming Guide](#) description of the later Graphics machines says “it is no longer necessary to keep track of ‘box boundaries’ when drawing lines”, so the Alpha machines must have had *some* limit. And it is also implied by how the largest single display subroutine, called “CRUNCH”, was written.

When the player collides with one of the other cars, the status message at the bottom of the screen is replaced by the word “CRUNCH!” surrounded by a cartoony explosion:



This graphic is drawn by the “CRUNCH” display subroutine with the following steps:

- Before calling CRUNCH, D LXA and D LYA instructions are used to set the beam at the location of the blue circle.
- Increment mode is used to draw the letters “CR” and the left side of the explosion.
- The LSBs are reset (returning the beam to the blue circle).
- The X MSB is incremented 4 times (with 4 D IXM instructions), moving the beam position to the red circle.
- Increment mode is used with the beam off to move to the base of the exclamation point.
- The letters “UNCH!” are written in reverse order and then the right side of the explosion is drawn, completing the graphic.

I think this tells us a few things.

- There is a limit to how far increment mode can move from the starting point. Otherwise you would just write the entire word “CRUNCH!” and then draw the explosion around it without having to stop and increment the MSBs at any point.
- The bounding box extends from at least from -22 to + 26 in the X-direction and from -22 to +24 in the Y-direction. Otherwise the graphic couldn’t be drawn.
- If the LSBs were updated to track the position of the beam in increment mode, this would require at least 48 unique position values (22+26), which would require more than the 5 bits available in the screen address.
- The bounding box is fairly symmetrical around the starting point and probably can’t extend more than 31 units in the negative direction. Otherwise there was no reason I can see for the programmer to have only incremented the MSBs to the center of the letters “UNCH!” and then used increment mode to move the beam the rest of the way. They could have saved instructions by incrementing the MSBs more times (to get the red circle closer to the bottom of the exclamation point). But it does save time if it keeps them from having to reset the beam a 3rd time to finish the graphic.

CONCLUSION:

There’s no way that I can see to deduce a perfect answer here, but my best guess is that there were no dedicated screen position registers (at least not ones updated in real time), and the bounding box for Increment Mode was somewhere between 27 and 31 units in both directions from the beam starting point. Maybe the equivalent of a 6-bit signed integer ([-32, +31] or perhaps a nonstandard [-31, +32]). But I think the easiest solution for emulating it (at least to start) is to just ignore the bounding box limitation and cross that bridge when we come to it. Conforming software will never hit the bounding box anyway, and if any software is found that runs into those limits, we might then be able to get additional clues to deduce how the system should behave when that happens.

Also, note that increment mode control bytes like “N”, “R”, “F” or a custom “A” byte can still “zero out” the LSBs in X or Y to return the beam to its initial position with a single instruction. Freeway does this in several places, so that functionality must still be implemented, even if there’s no actual register keeping track of the LSB position within the bounding box.

Question: How big is the whole grid?

The game's playfield is centered on the screen and is 80 grid spaces wide and 64 grid spaces tall. One question that needs to be answered for emulation is: how much of the actual screen does this take up? One possibility is that the playfield takes up nearly all of the screen, especially in the X-direction. This is tempting because the big grid is typically used to layout letters of text, and 80 across the width of the screen brings to mind the commonplace 80-column text mode that might have been a design target. (Though perhaps 1971 predates the era where 80-columns would have been a common convention. I don't really know.) Another possibility is that they used all 7 bits of the MSBs to divide the screen into a 128x128 outer grid with a 9x9 sub grid.

There's no way I can think of to deduce a definitive answer to this question from just Freeway alone. But thankfully, we have at least one other piece of software that I believe was written for the same Alpha hardware as Freeway.

In addition to the PDS-1D Programmers Guide, the other major piece of documentation that has survived is a Technical Manual for the PDS-1. Unfortunately, this manual doesn't say anything specific about how screen addressing works (perhaps because the design was still in flux), but I suspect that it was written for the Alpha hardware. For one thing, given the cover date of March 1970 the timeline makes sense for this. But also, on page VI-2.18, there is a [sample program](#) that prints the word "Hello" to the screen. This program has several quirks in common with Freeway, that cause me to suspect that they were written for the same machine:

- While the vector text "fonts" are not identical, the letterforms in both are designed to fit within a 6x8 bounding box which makes more sense for a 9x9 subgrid than 16x16.
- Both of them call the "D HVC" instruction at the beginning of the display routine, which the PDS-1D programming guide says is "required only on early PDS-1 models".
- Most importantly, both of them call "D LXA 4000" and "D LYA 4000" at the end of the display routine to "center" the beam while waiting for the next frame.

```
D LXA 4000 / CENTER BEAM TO MINIMIZE LOAD ON DEFLECTION AMPLIFIERS
D LYA 4000
```

```
2653      014000      1446  D LXA 4000 /CENTER BEAM AND PAI T
2654      024000      1447  D LYA 4000
```

(Note that the paper tape archived by Tom Uban, called "40tp_simpleDisplay.ptp", is not an exact copy of this program as it appears in the Technical Manual. On the paper tape, 5 lines of machine code have been changed to make it compatible with the later Graphics machine format. I'll say more about this in the next section.)

The important clue provided by this program is that, in order to print "HELLO" in the corner of the screen, it uses D LXA and D LYA to set the beam position to (0, 0). Given the center of the

screen is (4000, 4000), it must be the case that the MSBs use all 7 bits to divide the screen into a 128x128 large grid.

I should note that the comment at the top of the sample program says it's intended to print hello in the "Upper Left of Screen", but I'm pretty sure that (0, 0) should actually refer to the *lower* left. That's how it works for Graphics machines, and even within the sample program itself, the increment mode letter-forms assume that +Y = up. I guess it's not impossible that the address mode could use a different sign convention than the increment mode vectors, but that would be a pretty weird design choice. I suspect that this is just a case of changing the code and forgetting to update the comment. (Many such cases.)

CONCLUSION:

I'm pretty confident that the Alpha hardware used all 7 bits of the MSBs to divide the screen into a 128x128 grid with a 9x9 subgrid, resulting in a screen resolution of 1152 x 1152. That's actually a little more precise than the default Graphics machine resolution of 1024x1024, though if you set Scale to $\frac{1}{2}$ you can bump that up to 2048 x 2048.

Interestingly, this would imply that the maximum valid input to the D L_A instructions is not 7777_8 but 7750_8 (equivalent to 111111_{MSB} and 01000_{LSB})

Question: How does changing the global scale parameter affect things on an Alpha machine?

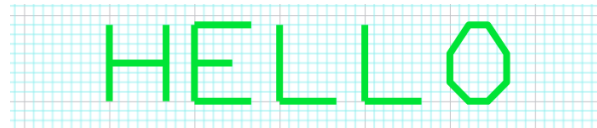
Freeway stays at Scale=1 throughout, so we can't use it to deduce anything about how the scale instruction worked on Alpha machines. But luckily, the sample Hello program (as originally written) uses Scale=2, and if you dig into the details, that actually tells us a lot.

Each of the four letter subroutines ends with an "F" command. This is a convenience function that the assembler uses to refer to a control byte in increment mode that does a collection of useful things:

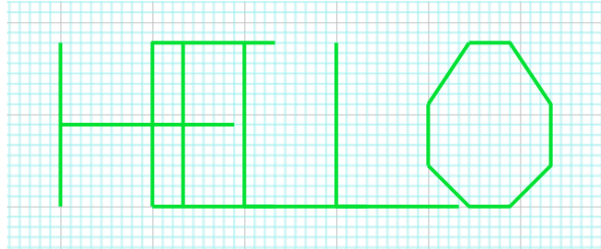
```
F           Exit increment mode, return from D JMS, add  
             one to X MSB, zero X and Y LSB. (171)
```

If each of your letter subroutines ends in F, then you just need to call letter subroutines one after the other to write text. Once it's done drawing the letter, it will zero the LSBs (to return the beam to where it started), increment the X MSB to move the beam into position for the next letter, and then return to the main drawing routine ready for the next letter to be called.

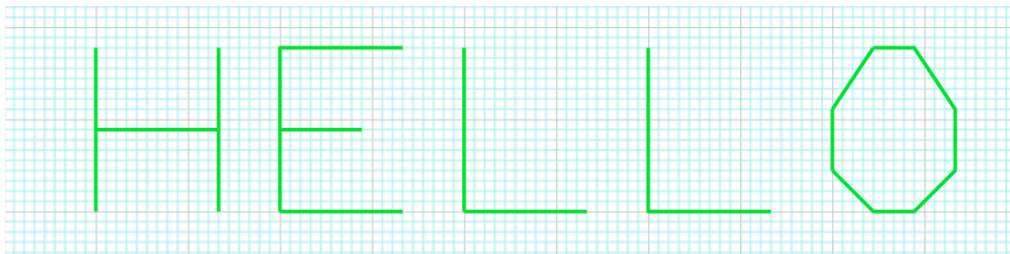
If scale was set equal to 1, then calling the H, E, L L, and O subroutines in that order would produce this:



But the sample program sets Scale=2. If the scale only affects the LSBs (as it does on a Graphics machine) you end up with this:



The F command only increments the X MSB once, so the double-size letters are 12 units wide, but only spaced 9 units apart and the letters bleed into each other. For this reason, I suspect that on an Alpha machine, setting Scale to 2 also doubles the size of the MSB increments. In that case, the sample program would just write the word Hello with the same proportions, but twice as large.



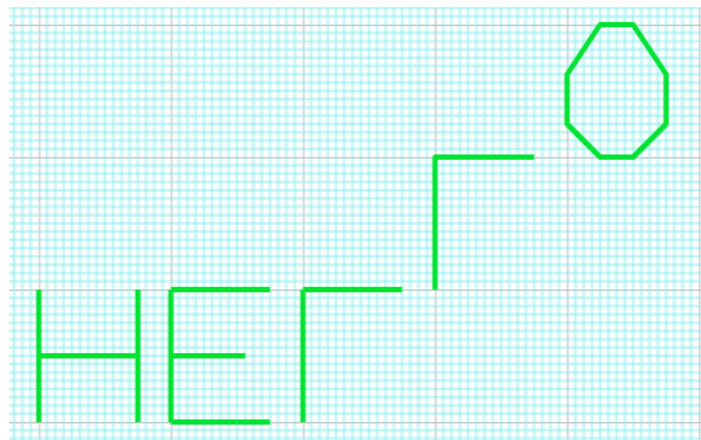
I think this makes some sense from a system design perspective. The utility of display subroutines is that they give you the ability to define reusable “sprites”. (I know technically they aren’t true sprites, but they’re sprite-like: defined repeatable images that you can easily move around the screen and scale up and down without much coding effort.) For an Alpha machine where the LSBs don’t automatically carry, any display subroutine larger than the bounding box will have to include manual MSB manipulation in addition to increment mode vectors. If D STS only scales the increment vectors but not the MSBs, then you can’t use it to scale these larger “sprites”.

Conversely, for Graphics machines, where the LSBs do carry, there’s generally no reason to mess around with the MSBs in the middle of a display subroutine. You can just stay in increment mode the whole time. Using D STS to scale the MSBs doesn’t make anything easier for you. But having it *not* scale the MSBs does get you something: no loss of precision when deciding where to place scaled sprites on the screen. So I think it makes some sense for IMLAC to have made this behavior change when they went from the Alpha design to the Graphic design.

Incidentally, this is a bit of a tangent, but I mentioned earlier that “40tp_simpleDisplay.ptp” had been modified to fit the Graphics machine format. Specifically, 5 lines of machine code were changed:

- The initial D LXA and D LYA instructions (at 0116 and 0117) were changed from (0,0) to (5000₈, 5000₈) which, because bits 4 and 5 are ignored, is equivalent to (1000₈, 1000₈). This moves the word HELLO to the center of the screen instead of the corner. I don’t know why they would have used 5000₈ instead of 1000₈, but it worked.
- The final D LXA and D LYA instructions to “center beam” (at 0127 and 0128) were also changed from (4000₈, 4000₈) to (5000₈, 5000₈).
- The last code change is the STS instruction (at 0120). It was changed from 004006 to 004005, i.e. changing the scale setting from 2 to 1.

The reason why the scale had to be changed is a little funny. If you were to compile the sample program as originally written and run it on a Graphics machine, the output would look like this:



This is because of how the Ls are drawn. The L subroutine starts by drawing the vertical line of the L in three increments (+3, +3, and +2). Then it resets the beam position by using a custom control byte (“A 1”) to do nothing but zero out the Y LSB, moving the beam back down to the starting point. Then it draws the horizontal line. If this is run at Scale=2 on a Graphics machine, the letter is exactly 16 units tall, which causes the Y LSB to just overflow and increment the MSB by 1. The LSB is already 0, so when the “A 1” byte is called, nothing happens! The beam then proceeds to draw the horizontal line of the L from it’s current position and continues on from there. If, instead, the L subroutine had stayed in increment mode to move back down and then draw the horizontal line, then the MSB would have decremented and it wouldn’t have been necessary to change the scale.

As it is, when you run the patched version on 40tp_simpleDisplay.ptp, it ends up looking like this, with the awkwardly-spaced text caused by writing letters designed for a 9x9 subgrid on a 16x16. I suppose that was the best the patcher thought they could do without redesigning all the letterforms.



CONCLUSION

My best guess for how D STS worked: Setting the global scale parameter equal to a value other than 1 should also scale the MSB's whenever they are incremented or decremented by a dedicated instruction or by an increment-mode control byte. However, the behavior of how D LXA or D LXY interpret a screen address shouldn't (I think) be affected.